

Phase 4.2: Optimization Passes

Overview

The optimization framework provides a flexible infrastructure for implementing and composing compiler optimizations. Each optimization pass operates on the BDI graph representation and can be combined in a pipeline for iterative optimization.

Architecture

OptimizationPassBase

Base class for all optimization passes:

- Inherits from `GraphVisitor` for graph traversal
- Tracks whether the graph was modified
- Provides `run()` method that returns true if changes were made

OptimizationEngine

Orchestrates multiple optimization passes:

- Maintains a pipeline of passes
- Runs passes iteratively until convergence or max iterations
- Supports fixed-point iteration for interdependent optimizations

Optimization Passes

1. Constant Folding

Purpose: Evaluate constant expressions at compile time.

Algorithm:

1. Identify operations with all constant operands
2. Compute result at compile time
3. Replace operation with constant result
4. Remove dead computation nodes

Examples:

- `2 + 3` → `5`
- `10 * 0` → `0`
- `x + 0` → `x`
- `x * 1` → `x`

Benefits:

- Reduces runtime computation
- Enables further optimizations (dead code elimination)
- Simplifies control flow

2. Dead Code Elimination (DCE)

Purpose: Remove computations that don't affect program output.

Algorithm:

1. Mark all nodes reachable from outputs (live nodes)
2. Remove unmarked nodes (dead code)
3. Update graph connectivity

Types of Dead Code:

- Unused variable definitions
- Unreachable code blocks
- Redundant computations
- Side-effect-free operations with unused results

Benefits:

- Reduces code size
- Improves cache utilization
- Simplifies further analysis

3. Common Subexpression Elimination (CSE)

Purpose: Eliminate redundant computations by reusing previously computed values.

Algorithm:

1. Build expression map: expression $\rightarrow$ node IDs
2. For expressions with multiple occurrences:
 - Keep first occurrence (canonical)
 - Replace other occurrences with reference to canonical
3. Handle commutativity ($a+b = b+a$)

Expression Key:

- Operation type
- Operand node IDs (sorted for commutative ops)

Benefits:

- Reduces redundant computation
- Decreases register pressure
- Enables value numbering

4. Loop Invariant Code Motion (LICM)

Purpose: Move loop-invariant computations outside loops.

Algorithm:

1. Detect loops using back-edge analysis
2. Identify loop-invariant instructions:
 - All operands defined outside loop, OR
 - All operands are themselves loop-invariant
3. Hoist invariant instructions to loop preheader
4. Ensure hoisting is safe (no side effects, dominates all uses)

Example:

```

Before:
loop:
    x = a * b    // a, b loop-invariant
    y = x + i
    i = i + 1
    if i < n goto loop

```

```

After:
x = a * b    // Hoisted
loop:
    y = x + i
    i = i + 1
    if i < n goto loop

```

Benefits:

- Reduces work per iteration
- Significant speedup for inner loops
- Enables further optimizations

5. Function Inlining

Purpose: Replace function calls with function body to eliminate call overhead.

Algorithm:

1. Identify inline candidates based on policy
2. Copy function body to call site
3. Remap parameters to arguments
4. Remap return values to call result
5. Update control flow

Inline Policy:

- `max_inline_size` : Maximum function size to inline
- `max_inline_depth` : Maximum inlining depth (prevent infinite recursion)
- `small_function_threshold` : Always inline very small functions
- Additional heuristics: call frequency, optimization opportunities

Benefits:

- Eliminates call overhead
- Enables interprocedural optimizations
- Improves instruction cache locality

Drawbacks:

- Increases code size
- May hurt instruction cache if overused

Usage Example

```
#include "OptimizationEngine.hpp"
#include "ConstantFolding.hpp"
#include "DeadCodeElimination.hpp"
#include "CommonSubexpressionElimination.hpp"

// Create optimization pipeline
OptimizationEngine engine;
engine.addPass(std::make_unique<ConstantFolding>());
engine.addPass(std::make_unique<CommonSubexpressionElimination>());
engine.addPass(std::make_unique<DeadCodeElimination>());

// Run optimizations
BDIGraph graph = /* ... */;
bool changed = engine.run(graph, 10); // Max 10 iterations

// Optimizations run until convergence or max iterations
```

Pass Ordering

Optimal pass ordering is crucial for effectiveness:

1. **Early passes:**
 - Constant folding (enables other optimizations)
 - CSE (reduces redundancy)
2. **Middle passes:**
 - LICM (loop optimizations)
 - Function inlining (enables interprocedural opts)
3. **Late passes:**
 - Dead code elimination (cleanup)
 - Final constant folding

Iterative Optimization

Many optimizations enable each other:

- Constant folding → Dead code elimination
- Inlining → Constant folding → CSE
- LICM → Dead code elimination

The engine runs passes iteratively until:

- No pass makes changes (convergence)
- Maximum iteration count reached

Implementation Details

Graph Modification Tracking

- Each pass sets `graph_modified_` flag when making changes
- Engine checks flag to determine if another iteration is needed

Safety Guarantees

- Passes preserve graph validity
- Type information maintained
- Control flow consistency enforced

Performance Considerations

- Passes use efficient data structures (hash maps, sets)
- Incremental updates where possible
- Early termination when no changes possible

Testing

150+ unit tests covering:

- Individual pass correctness
- Pass combinations and interactions
- Iterative optimization convergence
- Edge cases (empty graphs, single nodes)
- Performance with large graphs
- Optimization effectiveness metrics

Metrics

Track optimization effectiveness:

- Number of nodes eliminated
- Number of operations folded
- Code size reduction
- Estimated performance improvement

Future Enhancements

Potential additional passes:

- Strength reduction (replace expensive ops with cheaper ones)
- Loop unrolling
- Vectorization
- Alias analysis
- Interprocedural constant propagation
- Profile-guided optimization

References

1. Muchnick, "Advanced Compiler Design and Implementation" (1997)
2. Cooper & Torczon, "Engineering a Compiler" (2011)
3. Aho et al., "Compilers: Principles, Techniques, and Tools" (2006)